Day 18. Advanced implementation of multiclass classification

Improved implementation of softmax

Numerical Roundoff Errors

option 1

$$x = \frac{2}{10{,}000}$$

option 2

$$x = \underbrace{\left(1 + \frac{1}{10{,}000}\right)} - \underbrace{\left(1 - \frac{1}{10{,}000}\right)}$$

```
In [1]: x1 = 2.0 / 10000
        print(f"{x1:.18f}") # print 18 digits to the right of decimal point

        0.000200000000000000
```

```
In [2]: x2 = 1 + (1/10000) - (1 - 1/10000)
        print(f"{x2: .18f}")

        0.000199999999999978
```

Numerical Roundoff Errors

More numerically accurate implementation of logistic loss:

$1 + \frac{1}{10{,}000}$ $\quad 1 - \frac{1}{10{,}000}$

Logistic regression:

$$a = g(z) = \frac{1}{1 + e^{-z}}$$

```
model = Sequential([
  Dense(units=25, activation='relu'),
  Dense(units=15, activation='relu'),
  Dense(units=1, activation='sigmoid')
                 ])
model.compile(loss=BinaryCrossEntropy() )
```

'linear'

Original loss

$$loss = -y\log(a) - (1-y)\log(1-a)$$

```
model.compile(loss=BinaryCrossEntropy(from_logits=True) )
```

More accurate loss (in code)

$$loss = -y\log\left(\frac{1}{1+e^{-z}}\right) - (1-y)\log\left(1 - \frac{1}{1+e^{-z}}\right)$$

similarly, to improve numerical accuracy of softmax regression, we can use the way.

More numerically accurate implementation of softmax

Softmax regression

$$(a_1, \dots, a_{10}) = g(z_1, \dots, z_{10})$$

$$\text{Loss} = L(\vec{a}, y) = \begin{cases} -\log a_1 & \text{if } y = 1 \\ \vdots & \\ -\log a_{10} & \text{if } y = 10 \end{cases}$$

```
model = Sequential([
  Dense(units=25, activation='relu'),
  Dense(units=15, activation='relu'),
  Dense(units=10, activation='softmax')
                  ])
```

'linear'

~~model.compile(loss=SparseCategoricalCrossEntropy()~~)

More Accurate

$$L(\vec{a}, y) = \begin{cases} -\log \dfrac{e^{z_1}}{e^{z_1} + \dots + e^{z_{10}}} & \text{if } y = 1 \\ \vdots & \\ -\log \dfrac{e^{z_{10}}}{e^{z_1} + \dots + e^{z_{10}}} & \text{if } y = 10 \end{cases}$$

```
model.compile(loss=SparseCategoricalCrossEntropy(from_logits=True))
```

MNIST (more numerically accurate)

model

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
model = Sequential([
  Dense(units=25, activation='relu'),
  Dense(units=15, activation='relu'),
  Dense(units=10, activation='linear') ])
```

loss

```
from tensorflow.keras.losses import
  SparseCategoricalCrossentropy
model.compile(...,loss=SparseCategoricalCrossentropy(from_logits=True)
```

fit

```
model.fit(X,Y,epochs=100)
```

predict

```
logits = model(X)
f_x = tf.nn.softmax(logits)
```

not a

logistic regression (more numerically accurate)

model

```
model = Sequential([
  Dense(units=25, activation='sigmoid'),
  Dense(units=15, activation='sigmoid'),
  Dense(units=1, activation='linear')
                 ])
from tensorflow.keras.losses import
  BinaryCrossentropy
```

loss

```
model.compile(..., BinaryCrossentropy(from_logits=True))
```

fit

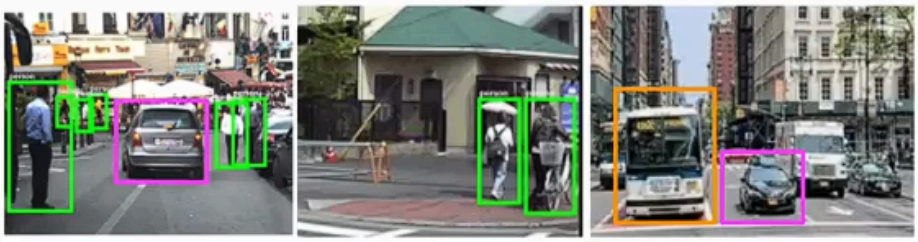
```
model.fit(X,Y,epochs=100)
```

predict

```
logit = model(X)
f_x = tf.nn.sigmoid(logit)
```

Clssification with multiple outputs

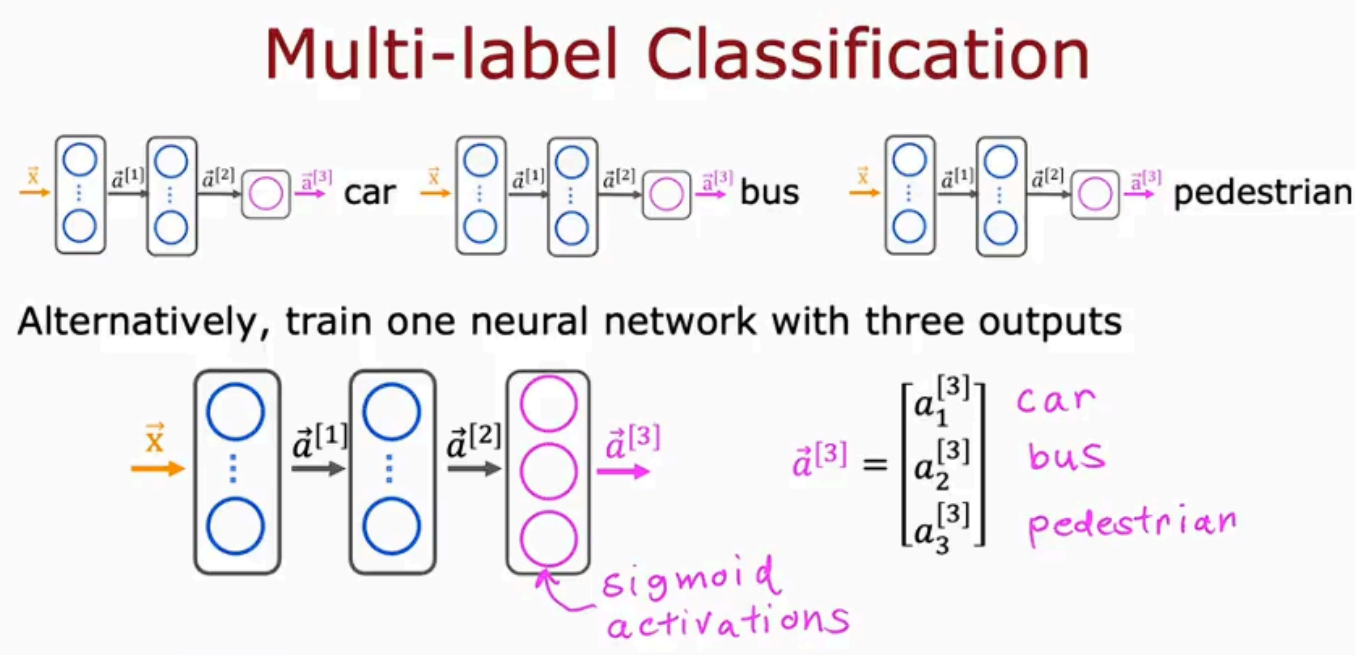
Multi-label Classification



Is there a car?	yes	no	yes
Is there a bus?	no	no	yes
Is there a pedestrian	yes	yes	no

to solve this kind of problem, one direct way is to use three neural networks to decide three outputs.

or, actually, we can train one neural network with three outputs.



Output Handling

Notice that in the preferred model, the outputs are not probabilities, but can range from large negative numbers to large positive numbers. The output must be sent through a softmax when performing a prediction that expects a probability. Let's look at the preferred model outputs:

In [11]:
```
p_preferred = preferred_model.predict(X_train)
print(f"two example output vectors:\n {p_preferred[:2]}")
print("largest value", np.max(p_preferred), "smallest value", np.min(p_preferred))
```

```
two example output vectors:
 [[ -1.78  -1.54   3.36  -0.62]
 [ 10.11   4.3   -0.28 -10.31]]
largest value 16.239902 smallest value -15.256491
```

The output predictions are not probabilities! If the desired output are probabilities, the output should be be processed by a softmax.

In [12]:
```
sm_preferred = tf.nn.softmax(p_preferred).numpy()
print(f"two example output vectors:\n {sm_preferred[:2]}")
print("largest value", np.max(sm_preferred), "smallest value", np.min(sm_preferred))
```

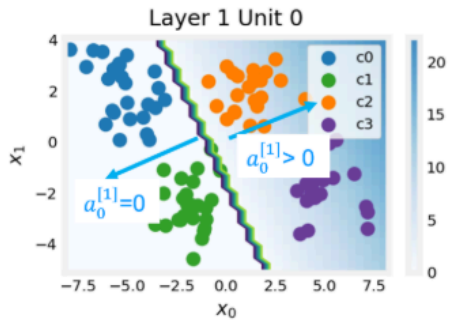
```
two example output vectors:
 [[5.65e-03 7.19e-03 9.69e-01 1.80e-02]
 [9.97e-01 2.99e-03 3.07e-05 1.35e-09]]
largest value 0.99999964 smallest value 2.0955116e-14
```

in the lab we can get more insight on what activation function can do and what the layers and units are doing.

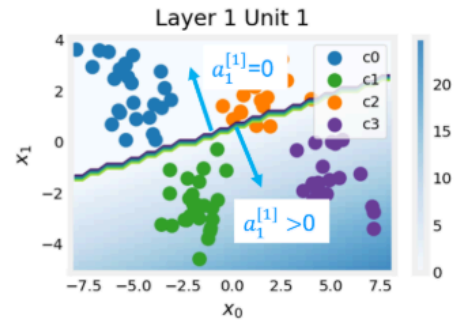
Explanation

Layer 1

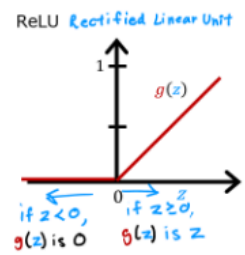
$$a_0^{[1]} = relu(w_{00}x_0 + w_{01}x_1)$$



$$a_1^{[1]} = relu(w_{10}x_0 + w_{11}x_1)$$

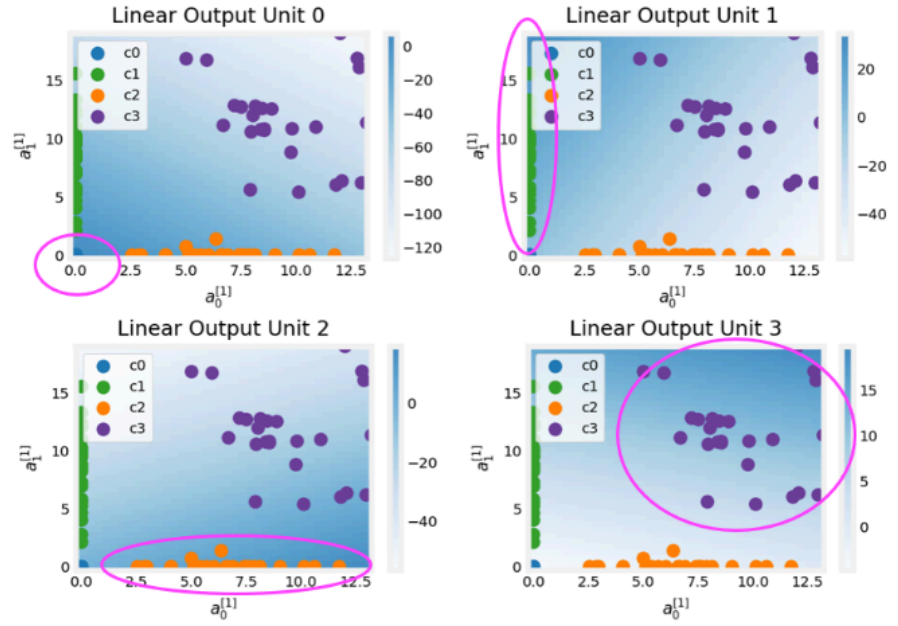


These plots show the function of Units 0 and 1 in the first layer of the network. The inputs are (x_0, x_1) on the axis. The output of the unit is represented by the color of the background. This is indicated by the color bar on the right of each graph. Notice that since these units are using a ReLu, the outputs do not necessarily fall between 0 and 1 and in this case are greater than 20 at their peaks. The contour lines in this graph show the transition point between the output, $a_j^{[1]}$ being zero and non-zero. Recall the graph for a ReLu : The contour line in the graph is the inflection point in the ReLu.



Unit 0 has separated classes 0 and 1 from classes 2 and 3. Points to the left of the line (classes 0 and 1) will output zero, while points to the right will output a value greater than zero. Unit 1 has separated classes 0 and 2 from classes 1 and 3. Points above the line (classes 0 and 2) will output a zero, while points below will output a value greater than zero. Let's see how this works out in the next layer!

Layer 2, the output layer



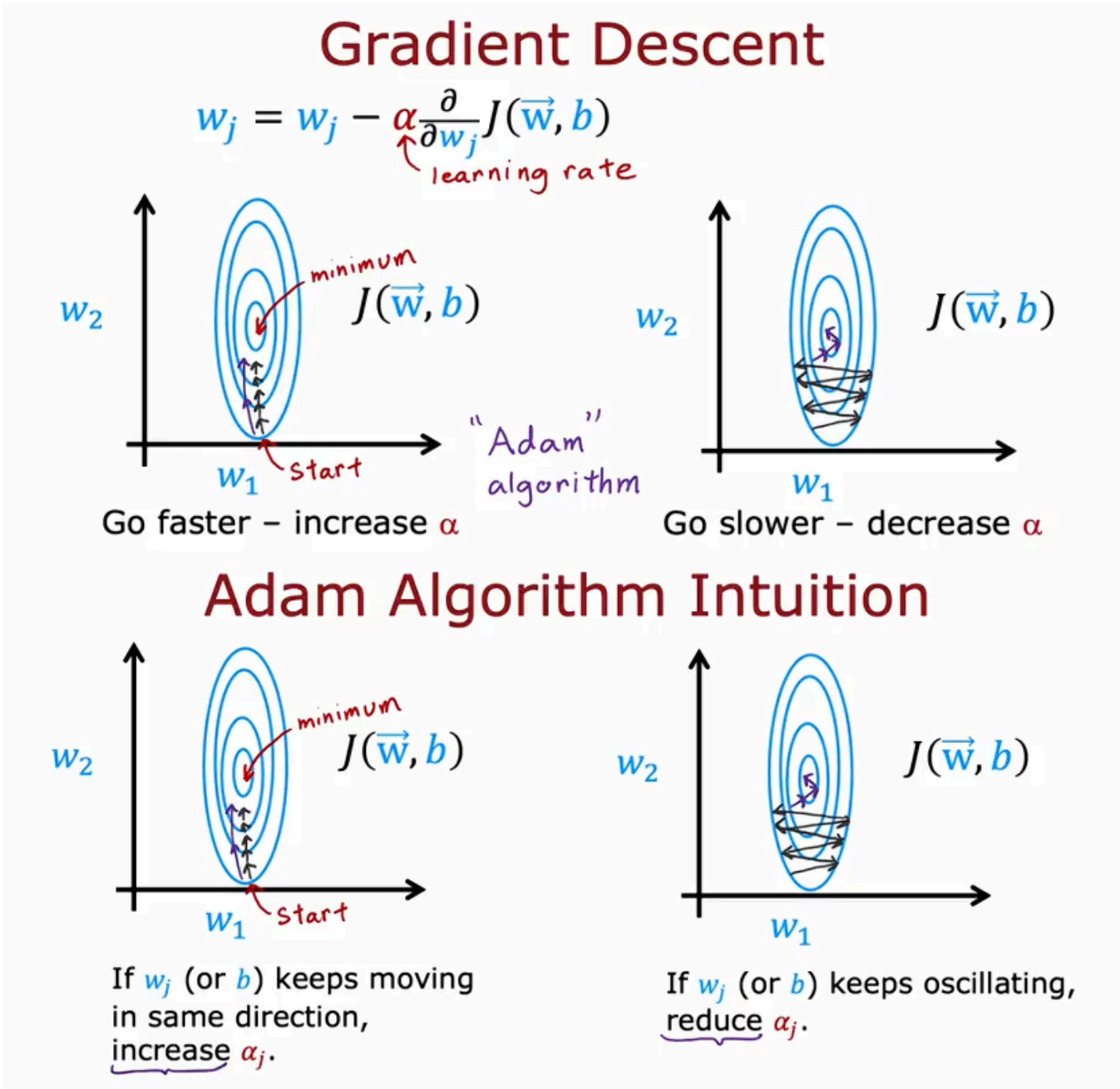
The dots in these graphs are the training examples translated by the first layer. One way to think of this is the first layer has created a new set of features for evaluation by the 2nd layer. The axes in these plots are the outputs of the previous layer $a_0^{[1]}$ and $a_1^{[1]}$. As predicted above, classes 0 and 1 (blue and green) have $a_0^{[1]} = 0$ while classes 0 and 2 (blue and orange) have $a_1^{[1]} = 0$. Once again, the intensity of the background color indicates the highest values. Unit 0 will produce its maximum value for values near (0,0), where class 0 (blue) has been mapped.
Unit 1 produces its highest values in the upper left corner selecting class 1 (green).
Unit 2 targets the lower right corner where class 2 (orange) resides.
Unit 3 produces its highest values in the upper right selecting our final class (purple).

One other aspect that is not obvious from the graphs is that the values have been coordinated between the units. It is not sufficient for a unit to produce a maximum value for the class it is selecting for, it must also be the highest value of all the units for points in that class. This is done by the implied softmax function that is part of the loss function (`SparseCategoricalCrossEntropy`). Unlike other activation functions, the softmax works across all the outputs.

You can successfully use neural networks without knowing the details of what each unit is up to. Hopefully, this example has provided some intuition about what is happening under the hood.

Advanced optimization

gradient descent is an optimization algorithm that is widely used in ML, however, there are some other optimization algorithms outh there and some are even better than gradient descent.



the exact implementation of Adam is a bit complicated and not the scope of the course, and will

be covered in the deep learning courses.

MNIST Adam

model

```
model = Sequential([
        tf.keras.layers.Dense(units=25, activation='sigmoid'),
        tf.keras.layers.Dense(units=15, activation='sigmoid'),
        tf.keras.layers.Dense(units=10, activation='linear')
])
```

compile

$\alpha = 10^{-3} = 0.001$

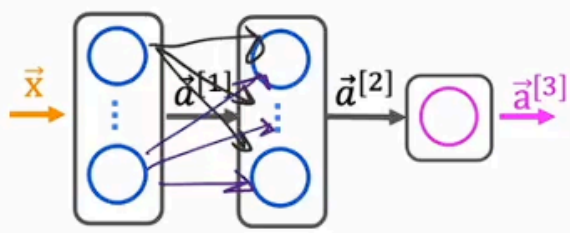
```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
  loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True))
```

fit

```
model.fit(X,Y,epochs=100)
```

Additional layer types

Dense Layer

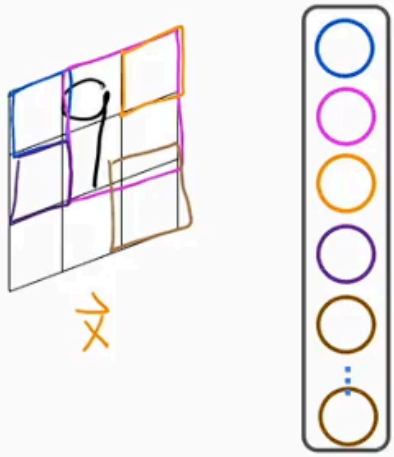


Each neuron output is a function of all the activation outputs of the previous layer.

$$\vec{a}_1^{[2]} = g\left(\vec{w}_1^{[2]} \cdot \vec{a}^{[1]} + b_1^{[2]}\right)$$

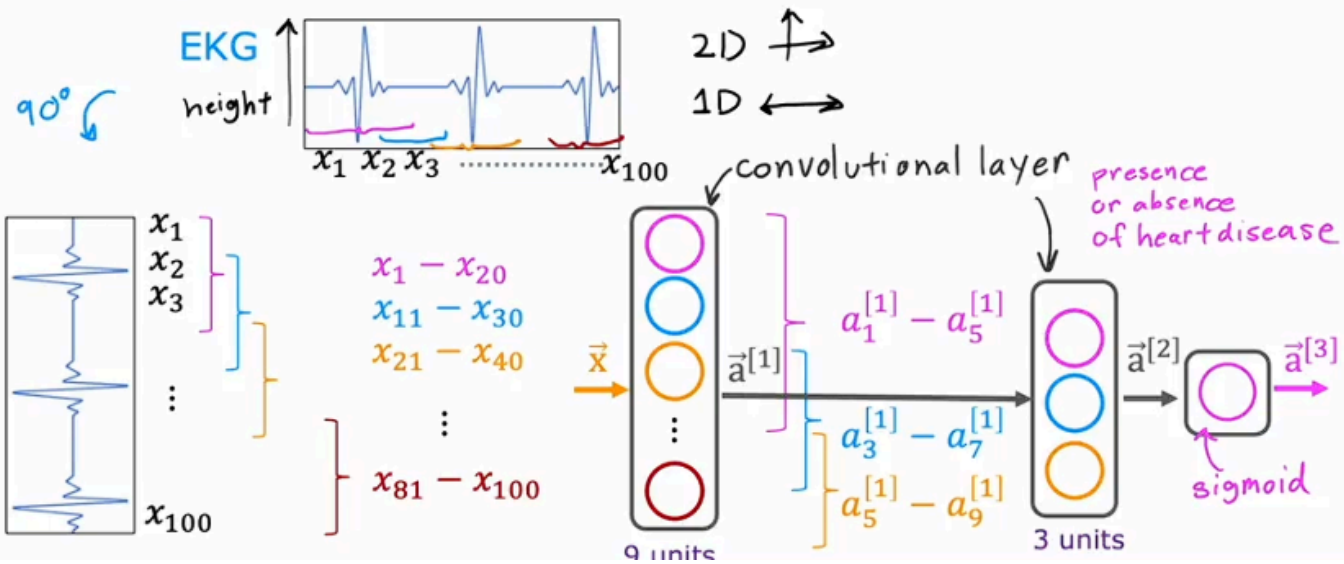
for example convolutional layer

Convolutional Layer



Each neuron only looks at part of the previous layer's outputs.

Why?

- Faster computation
- Need less training data (less prone to overfitting)

Convolutional Neural Network